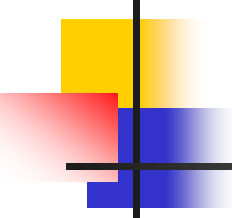


Collections

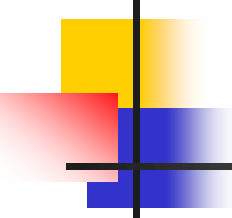


Dr. Khadim Dramé
Khadim.drame@univ-zig.sn
Département Informatique
Université Assane Seck de Ziguinchor



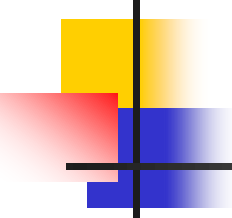
Plan du cours

- Introduction
- Itérateurs
- Interface **Collection**
- Listes
- Ensembles
- Classe **Collections**
- Interface **Comparable**
- Tableaux associatifs



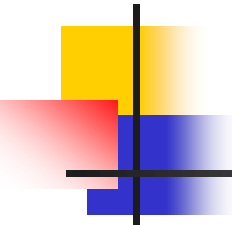
Introduction

- **Collection** : structure de données pour regrouper un ensemble d'objets dans une seule entité
- Certaines collections acceptent les **doublons**, d'autres non
- Certaines sont **ordonnées** (tableaux dynamiques, listes chaînées), d'autres pas (ensembles)



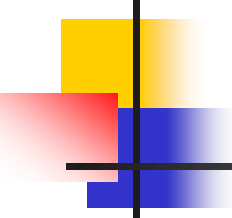
Introduction

- Limites des tableaux
 - taille fixe
 - ajout et suppression d'éléments difficile
 - éléments de même type
- Collections
 - pas de limite sur la taille
 - opérations d'ajout et de suppression simplifiées
 - possibilité de contenu hétérogène



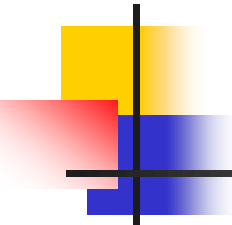
Introduction

- API Collections depuis Java 1.2 (1998)
 - Ensemble de classes et interfaces dans Java SE
- Classes et interfaces **génériques** (Java 1.5)
- Plusieurs types de collections
 - Listes (**List**)
 - Files (**Queue**)
 - Ensembles (**Set**)
 - Tableaux associatifs (**Map**)
- Classes et interfaces dans le package **java.util**



Itérateurs

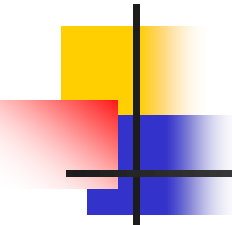
- **Itérateur** : objet pour parcourir les éléments d'une collection un à un
- Deux types d'itérateurs
 - **Itérateurs unidirectionnels**
 - le parcours se fait du début vers la fin
 - **Itérateurs bidirectionnels**
 - le parcours peut se faire dans les deux sens



Itérateurs unidirectionnels

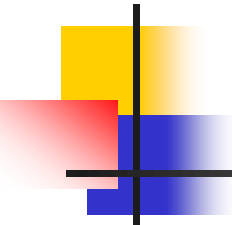
- **Collection** a une méthode **iterator()** renvoyant un itérateur unidirectionnel (**Iterator**) avec les méthodes
 - **next** pour obtenir l'objet désigné et avancer l'itérateur
 - **hasNext** pour vérifier si l'itérateur est ou non à la fin de la collection
 - **remove** pour supprimer le dernier objet renvoyé par **next**
- **Exemple**
 - Parcourir les éléments d'une collection c

```
Iterator iter = c.iterator(); //renvoie un objet implémentant Iterator
while (iter.hasNext()){ //vérifie si l'itérateur n'est pas en fin de collection
    Object o = iter.next(); //fournit l'élément désigné par l'itérateur
                        //et avance l'itérateur à la position suivante
}
```



Itérateurs bidirectionnels

- Parcours de certaines collections (listes chaînées, vecteurs) dans les deux sens
- Ces collections ont une méthode `listIterator()` renvoyant un itérateur bidirectionnel (`ListIterator`) avec les méthodes
 - `next`, `hasNext`, `remove` héritées de `Iterator`
 - `previous` et `hasPrevious`
 - `add` pour ajouter un élément à la position courante et avancer
 - `set` pour remplacer l'élément courant (position courante pas modifiée)



Itérateurs bidirectionnels

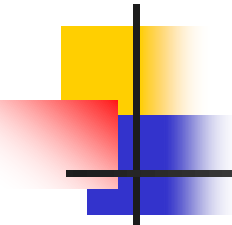
■ Exemples

■ Parcourir les éléments d'une liste lst

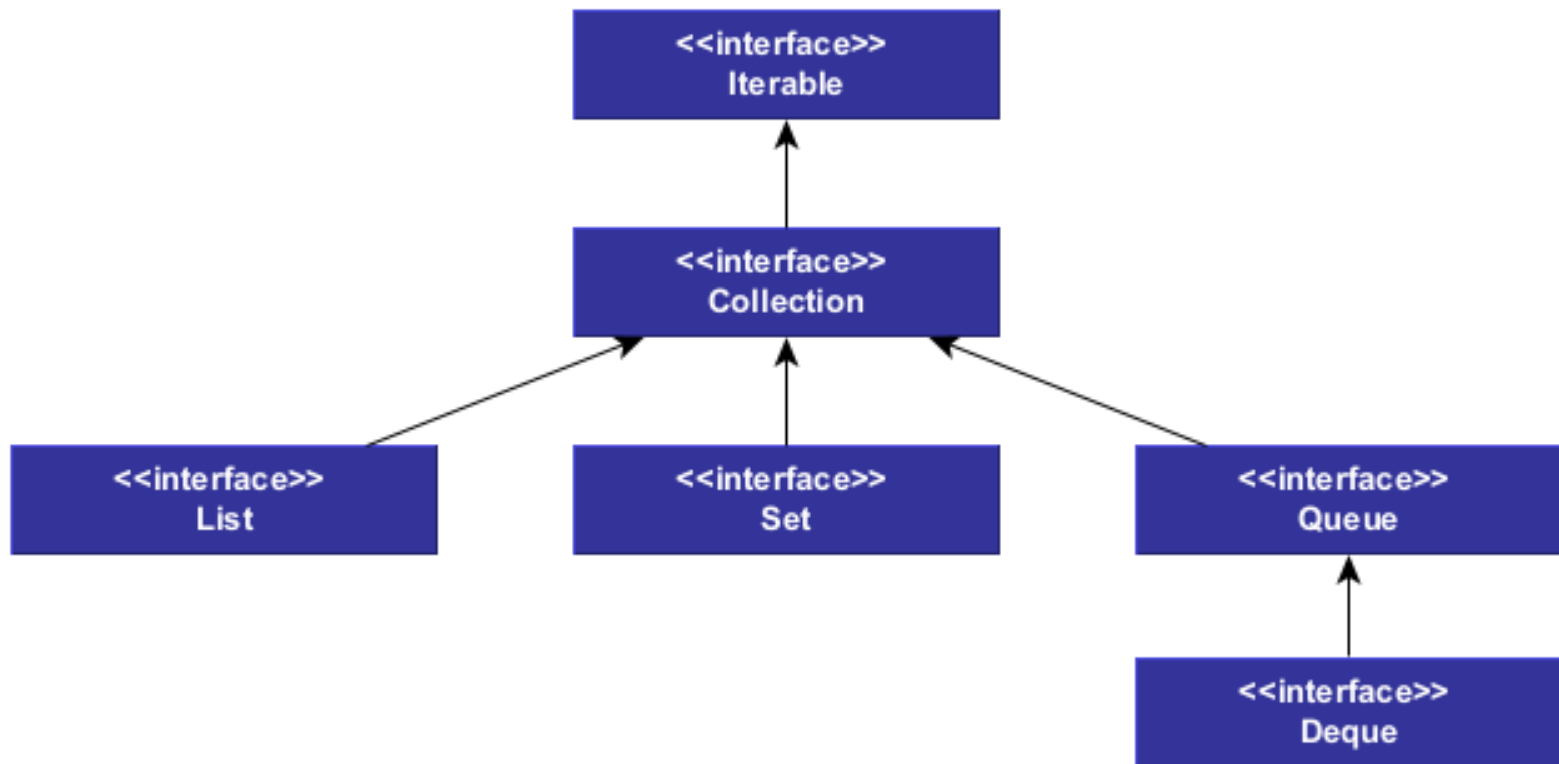
```
ListIterator iter = lst.listIterator(lst.size); // position courante : fin de liste
while(iter.hasPrevious()){
    Object o = iter.previous()
    //utilisation de l'objet courant o
}
```

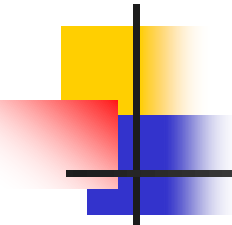
■ Ajouter un élément dans une liste lst

```
ListIterator iter = lst.listIterator();
iter.next(); // élément courant est le premier élément
iter.next(); // élément courant est le deuxième élément
iter.add(elem); //ajoute elem à la position courante (après le 2ème)
```



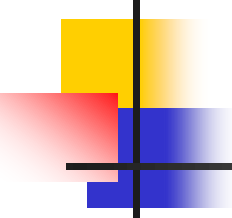
Interface Collection





Méthodes de l'interface Collection

- Méthodes communes aux collections implémentant l'interface **Collection**
 - **add(e)**: ajoute l'élément **e** et fournit **true** si l'opération s'est bien réalisée
 - **remove(e)**: supprime l'élément **e** de la collection
 - **addAll(c)**: ajoute tous les éléments de la collection **c**
 - **removeAll(c)**: supprime tout élément appartenant à la collection **c**
 - **retainAll(c)**: supprime tout élément n'appartenant pas à la collection **c**

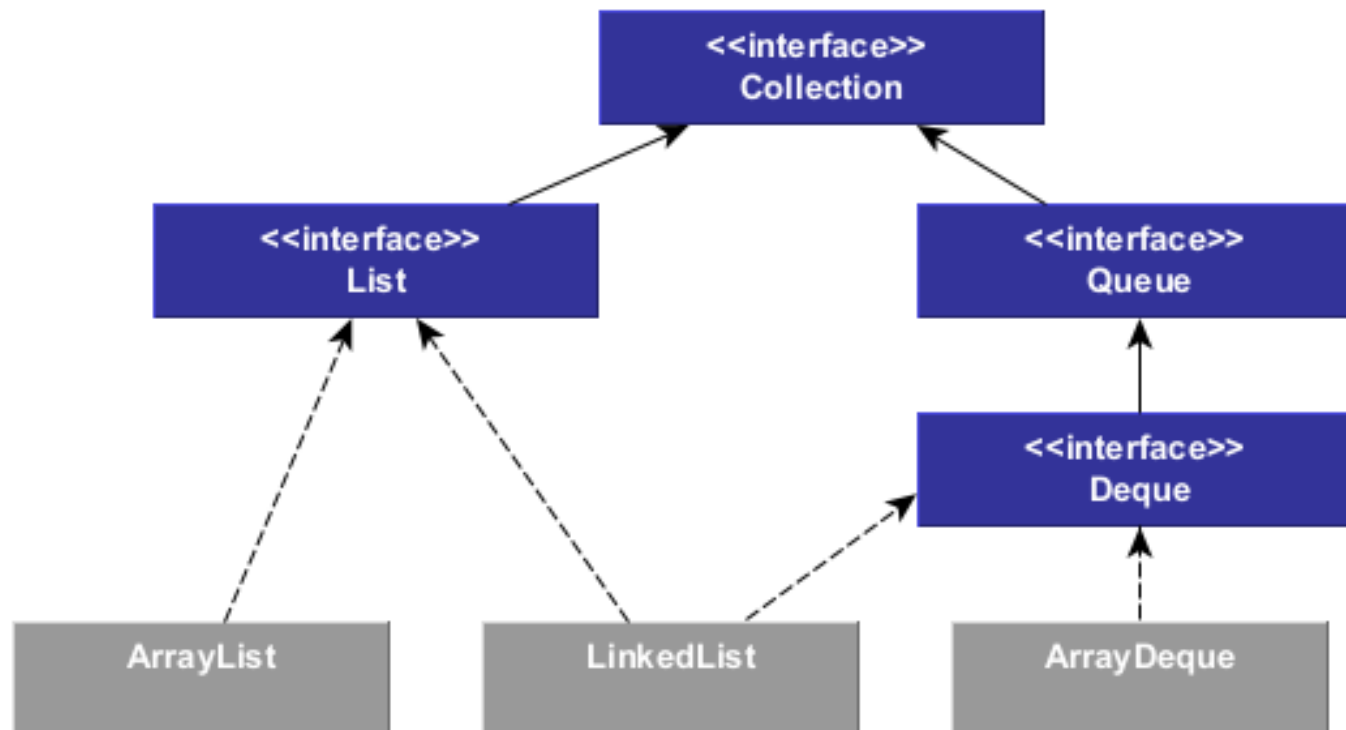


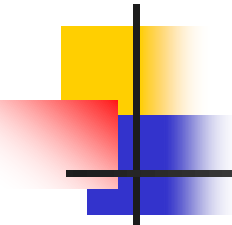
Méthodes de l'interface Collection

- Méthodes communes aux collections implémentant l'interface **Collection**
 - `size()`: fournit la taille de la collection
 - `isEmpty()`: vérifie si la collection est vide
 - `clear()`: supprime tous les éléments de la collection
 - `contains()`: vérifie si la collection contient un élément
 - `toString()`: décrit le contenu d'une collection
 - `iterator()` retourne un itérateur unidirectionnel

Listes

- **Liste** : collection ordonnée d'objets





Méthodes de l'interface List

- Opérations usuelles

- construction

- liste vide

- ```
List list = new ArrayList();
```

- à partir d'une collection c

- ```
List list = new ArrayList(c);
```

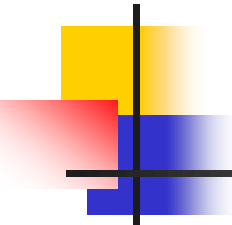
- parcours

- `Iterator` ou `for` ou `for each`

- ajout

- `add(e)`: ajoute l'élément **e** à la fin de la liste

- `add(i, e)`: ajoute l'élément **e** au rang **i**



Méthodes de l'interface List

■ Opérations usuelles

■ suppression

- `remove(e)`: supprime la 1^{ère} occurrence de l'élément **e**
- `remove(i)`: supprime l'élément au rang **i**
- `removeRange(i, j)`: supprime les éléments de rang **i** à **j**

■ modification

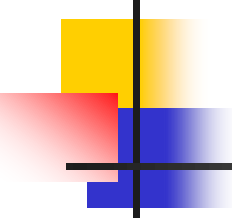
- `set(i, e)`: remplace l'élément au rang **i** par **e**

■ accès

- `get(i)`: donne l'élément de rang **i**

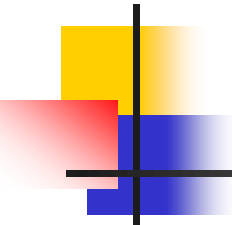
■ taille de la liste

- `size()`: donne le nombre d'éléments de la liste



Classe ArrayList (1/5)

- Manipulation de **tableaux dynamiques**
- Bonne **performance** pour l'**accès** en lecture aux éléments
- Opérations d'**ajout** et de **suppression** en temps linéaire
- Possibilité de réserver un espace mémoire pour n éléments lors de sa création



Classe ArrayList (2/5)

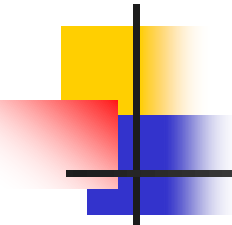
■ Exemples

- Créer un vecteur et y ajouter 5 entiers

```
List<Integer> vecteur = new ArrayList<Integer>();  
for (int i=1; i<=5; i++)  
    vecteur.add (new Integer(i)); // add(i) acceptée maintenant
```

- Afficher le contenu du vecteur

```
for (int i=0; i<vecteur.size(); i++)  
    System.out.print(vecteur.get(i) + " ");
```



Classe ArrayList (3/5)

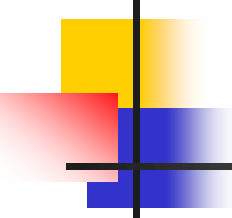
- Exemples

- Parcours avec un itérateur

```
Iterator iter = vecteur.iterator();  
while (iter.hasNext()){  
    System.out.print(iter.next() + " ");  
}
```

- Parcours avec for each

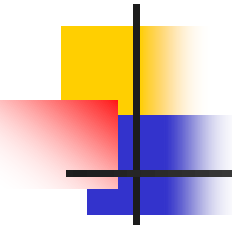
```
for (int n : vecteur){  
    System.out.println(n + " ");  
}
```



Classe ArrayList (4/5)

■ Exemple

```
public static void main(String[] args) {  
    List liste = new ArrayList();  
    liste.add(5);  
    liste.add("Bonjour ");  
    liste.add(10.5);  
    liste.remove(1);  
    liste.set(1, 20);  
    for (int i = 0; i < liste.size(); i++)  
        System.out.print(liste.get(i)+" ");  
}
```

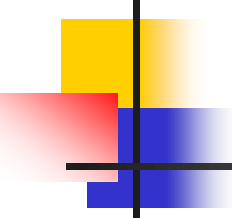


Classe ArrayList (5/5)

■ Exercice d'application

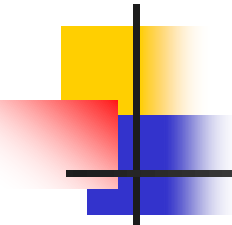
Écrire un programme qui

- demande à l'utilisateur de saisir une liste d'entiers
- demande à l'utilisateur de saisir une valeur à supprimer de la liste
- supprime la première occurrence (ensuite **toutes les occurrences**) de cette valeur de la liste
- affiche la liste après suppression



Classe LinkedList (1/4)

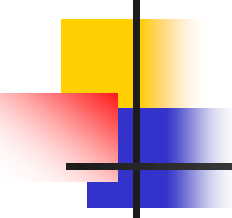
- Manipulation de **listes** doublement **chaînées**
- Possibilité de parcours avec un itérateur (bidirectionnel)
- Bonne **performance** pour les opérations d'**insertion** et de **suppression**
- **Accès** en lecture aux éléments en temps linéaire
- Possibilité de représenter des structures de type LIFO (Pile) et FIFO (File)



Classe LinkedList (2/4)

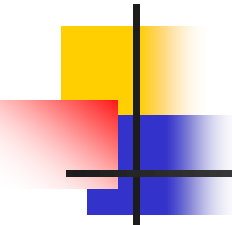
■ Exemple

```
public static void main(String[] args) {  
    LinkedList liste = new LinkedList();  
    liste.add(5);  
    liste.add("Bonjour ");  
    liste.add(10.5);  
    liste.addFirst("Bienvenue");  
    liste.addLast("Au revoir");  
    for (int i = 0; i < liste.size(); i++)  
        System.out.println("élément d'indice " + i + " = " + liste.get(i));  
}
```



Classe ArrayDeque

- Manipulation des **pires** (LIFO) et **files** (FIFO)
- Plus **performant** que **LinkedList** pour l'**insertion** et la **suppression** en **tête** et **fin** de liste
- Possibilité de réserver un espace mémoire pour n éléments lors de sa création
- Méthodes spécifiques (**Deque**)
 - **push**: empiler un élément
 - **peek**: lire l'élément de la tête sans l'enlever
 - **poll**: lire l'élément de la tête et l'enlever

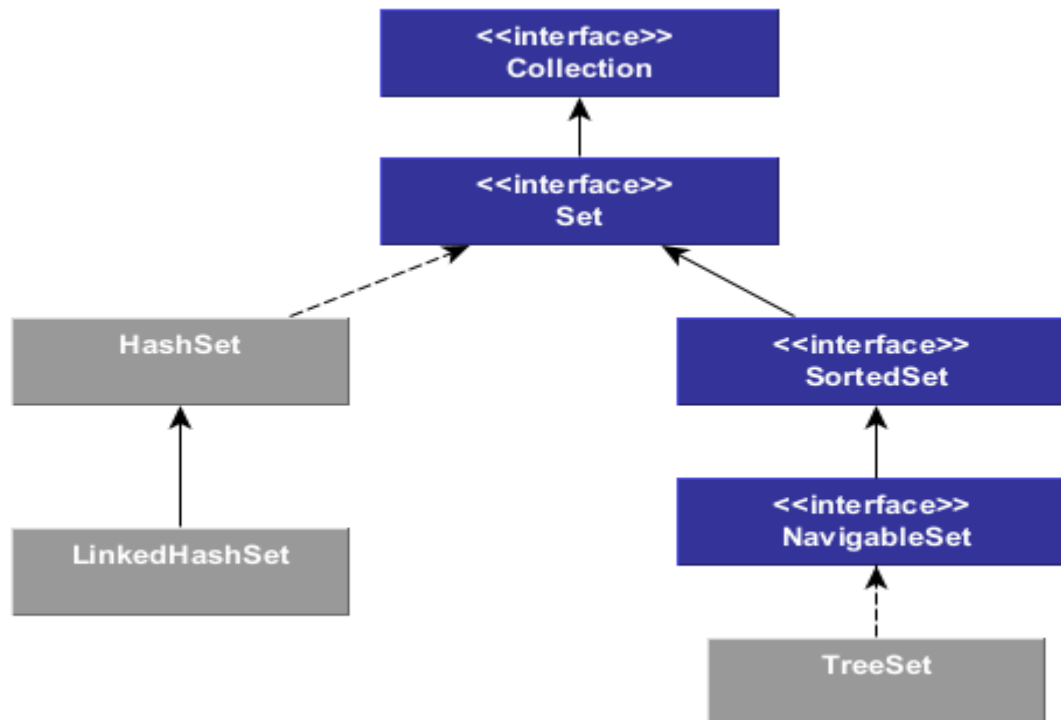


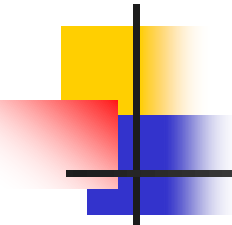
Classes Vector et Stack

- Incluses dans Java SE depuis la version 1.0
- **Vector** pour représenter les éléments d'une liste comme **ArrayList**
- **Stack** sous classe de **Vector** pour représenter les piles comme **ArrayDeque**
- Recommandations
 - utiliser **ArrayList** au lieu de **Vector**
 - utiliser **ArrayDeque** au lieu de **Stack**

Ensembles

- **Ensemble** : collection d'objets sans doublon





Méthodes de l'interface Set

- Opérations usuelles

- construction

- ensemble vide

- ```
Set set = new HashSet();
```

- à partir d'une collection c

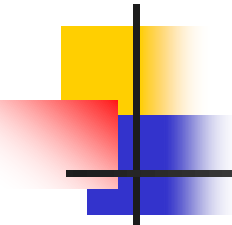
- ```
Set set = new HashSet(c);
```

- parcours

- `Iterator` ou `for each` ou `for`

- ajout

- `add(e)`: ajoute l'élément `e` à l'ensemble s'il n'y existe pas déjà



Méthodes de l'interface Set

- Opérations usuelles

- suppression

- `remove(e)`: supprime l'élément `e` de l'ensemble

- opérations ensemblistes

- **union**

- `set1.addAll(set2)`

- **intersection**

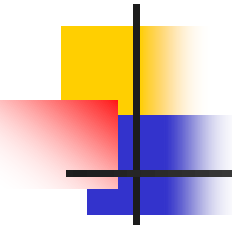
- `set1.retainAll(set2)`

- **différence**

- `set1.removeAll(set2)`

- **appartenance**

- `contains()`

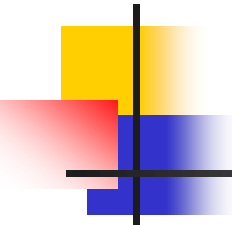


Classe HashSet

- Contrôle de l'unicité des éléments en utilisant une technique de hachage
- **Ordre** d'insertion des éléments **pas conservé**
- Plus **performant** (en temps) que **TreeSet** pour les opérations d'**ajout** et de **suppression**

```
Set<String> ensemble = new HashSet<String>();  
ensemble.add("élément 1");  
ensemble.add("élément 3");  
ensemble.add("élément 2");  
ensemble.add("élément 4");  
System.out.println(ensemble);
```

[élément 3, élément 2, élément 1, élément 4]

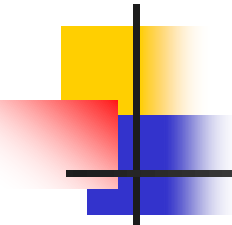


Classe LinkedHashSet

- Utilisation d'un code de hachage (comme [HashSet](#))
- Ordre de parcours des éléments = **ordre** de leur **insertion**
- Création à partir d'un autre set → **ordre** des éléments **maintenu**

```
Set<String> ensemble = new LinkedHashSet<String>();  
ensemble.add("élément 1");  
ensemble.add("élément 3");  
ensemble.add("élément 2");  
ensemble.add("élément 4");  
System.out.println(ensemble);
```

[élément 1, élément 3, élément 2, élément 4]

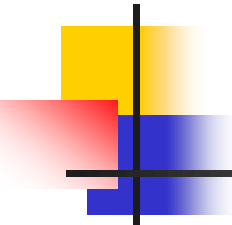


Classe TreeSet

- Contrôle de l'unicité des éléments en utilisant une liste triée en interne
- Les éléments sont de même type
- Ses éléments sont **triés**

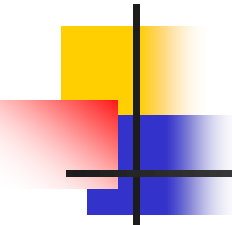
```
Set<String> ensemble = new TreeSet<String>();  
ensemble.add("élément 1");  
ensemble.add("élément 3");  
ensemble.add("élément 2");  
ensemble.add("élément 4");  
System.out.println(ensemble);
```

```
[élément 1, élément 2, élément 3, élément 4]
```



Classe Collections (1/2)

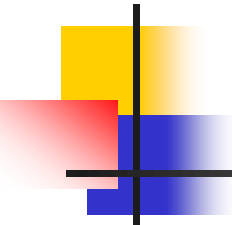
- Classe outil contenant des méthodes (statiques) pour manipuler des collections (List, Set)
 - **max**: renvoie le maximum d'une collection
 - **min**: renvoie le minimum d'une collection
 - **frequency**: la fréquence d'un élément dans une collection
 - **reverse**: inverse l'ordre des éléments d'une **liste**
 - **sort**: trie une **liste**
 - **fill**: remplace tous les éléments d'une **liste**
 - **unmodifiableList**: crée une liste non modifiable



Classe Collections (2/2)

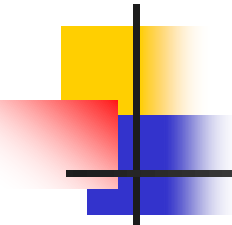
■ Exemple

```
import java.util.* ;
public class TestCollections {
    public static void main (String args[]){
        int nb[] = {4, 9, 2, 3, 8, 1, 3, 5} ;
        ArrayList tab = new ArrayList() ;
        for (int i=0; i<nb.length; i++)
            tab.add (new Integer(nb[i])) ;
        System.out.println ("tableau initial = " + tab);
        Collections.sort (tab) ;
        System.out.println ("tableau trié = " + tab);
        Collections.shuffle (tab) ;
        System.out.println ("tableau mélangé = " + tab);
        Collections.sort (tab, Collections.reverseOrder());
        System.out.println ("tableau trié décroissant = " + tab);
    }
}
```

Interface Comparable (1/2)

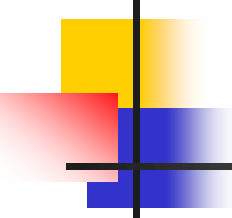
- Certaines collections nécessitent de **comparer** leurs éléments
- Les classes des éléments à comparer doivent implémenter l'interface **Comparable**
 - redéfinir la méthode public int **compareTo**(Object o) qui comparer l'objet courant **this** à l'objet **o** et renvoie un entier :
 - nul en cas d'égalité)
 - négatif si l'objet courant est inférieur à o
 - positif si l'objet courant est supérieur à o



Interface Comparable (2/2)

■ Exemple

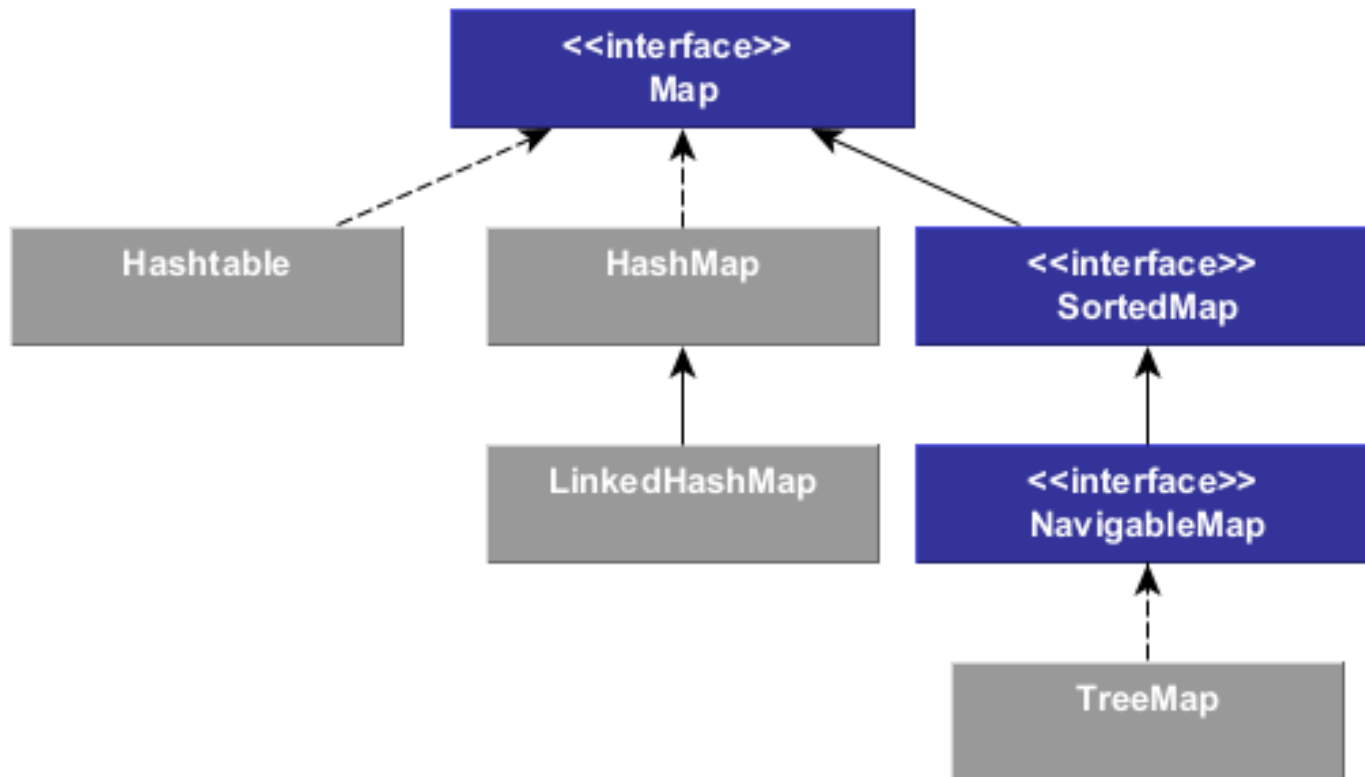
```
import java.util.* ;  
public class Personne implements Comparable {  
    public int compareTo(Object o) {  
        if (! (o instanceof Personne) ) throw new  
        ClassCastException("impossible de comparer un " +  
        o.getClass().getName() + " à une Personne");  
        Personne p = (Personne) o;  
        String s = this.nom + this.prenom;  
        return s.compareTo(p.nom + p.prenom )  
    }  
}
```

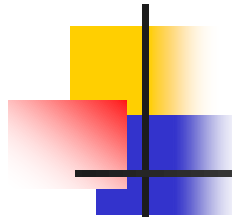


Tableaux associatifs

- Tableau associatif ou **dictionnaire** ou **map**
- Manipulation d'informations associant une **clé** à une **valeur**
- Exemples
 - Dictionnaire: chaque **mot** associé à une **définition**
 - Annuaire: chaque **nom** associé à un **numéro**
- Accès rapide à une clé et l'information associée
- Clés uniques

Interface Map





Méthodes de l'interface Map

■ Opérations usuelles

■ construction

- `HashMap h = new HashMap();` // table vide

■ ajout, suppression, modification (mise à jour)

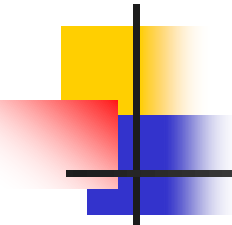
- `put(k, v)`: ajoute une paire ou remplace la valeur si la clé existe
- `remove(k)`: supprime la paire correspondant à la clé

■ accès

- `get(k)`: donne la valeur associée à la clé (null si clé absente)

■ appartenance

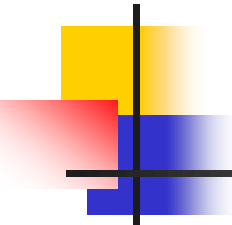
- `containsKey(k)`: vérifie si une clé est dans la table
- `Contains(v)`: vérifie si une valeur est dans la table



Méthodes de l'interface Map

- Opérations usuelles

- parcours d'une table de hachage
 - `entrySet()`: donne l'ensemble des paires de la table
 - parcourir cet ensemble (avec un itérateur ou for)
 - `getKey()`: récupère la clé d'une paire
 - `getValue()`: récupère la valeur d'une paire
 - `keySet()`: donne l'ensemble des clés de la table
 - parcourir cet ensemble et récupérer les valeurs correspondantes
 - `Values()`: donne une collection des valeurs de la table



Classes implémentant Map

■ **HashMap**

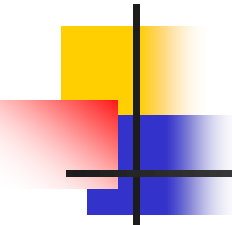
- contrôle de l'unicité des clés en utilisant un code de hachage
- **ordre** d'insertion des clés **pas conservé**
- plus **performant** que **TreeMap** pour l'opération d'**ajout**

■ **LinkedHashMap**

- contrôle de l'unicité des clés en utilisant un code de hachage
- ordre de parcours des clés = **ordre** de leur **insertion**
- création à partir d'un autre map → **ordre** des clés **maintenu**

■ **TreeMap**

- contrôle de l'unicité des clés en utilisant une liste triée en interne
- clés **triées**



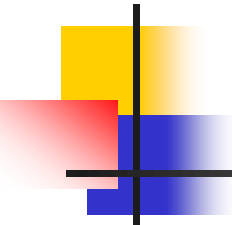
Tableaux associatifs

■ Exemples

```
Map<String, Integer> m1 = new HashMap<String, Integer>();  
m1.put("Pascal", 1);  
m1.put("Java", 2);  
m1.put("Python", 3);  
System.out.println(m1.keySet()+" "+m1.values()); // [Java, Pascal, Python] [2, 1, 3]
```

```
Map<String, Integer> m2 = new LinkedHashMap<String, Integer>();  
m2.put("Pascal", 1);  
m2.put("Java", 2);  
m2.put("Python", 3);  
System.out.println(m2.keySet()+" "+m2.values()); // [Pascal, Java, Python] [1, 2, 3]
```

```
Map<String, Integer> m3 = new TreeMap<String, Integer>();  
m3.put("Pascal", 1);  
m3.put("Java", 2);  
m3.put("Python", 3);  
System.out.println(m3.keySet()+" "+m3.values()); // [Java, Pascal, Python] [2, 1, 3]
```

Tableaux associatifs : application

■ Exercice d'application

Écrire un programme qui parcourt une liste et compte le nombre d'occurrences de chaque élément puis stocke l'élément avec sa fréquence dans un dictionnaire (map).

Exemple

```
List<Integer> list = Arrays.asList(2,5,10,3,2,10,5,10,2);
```

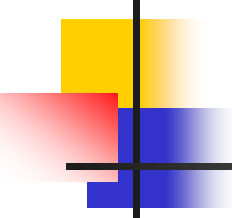
Le dictionnaire doit contenir les couples

(2, 3) //élément - fréquence

(3, 1)

(5, 2)

(10, 3)



Tableaux associatifs : application

```
public static void main(String[] args) {  
    List<Integer> list = Arrays.asList(2, 5, 10, 3, 2, 10, 5, 10, 2);  
    Map<Integer, Integer> map = new HashMap<Integer, Integer>();  
    for (int elt : list) {  
        if (map.containsKey(elt))  
            map.put(elt, map.get(elt)+1);  
        else  
            map.put(elt, 1);  
    }  
    for (Map.Entry<Integer, Integer> entry : map.entrySet()) {  
        System.out.println(entry.getKey() + " " + entry.getValue());  
    }  
}
```